

LP – Lógica de Programação

Prof. Didier Vega

E-mail: didier.vega@unifesp.br

Programação de computadores

Objetivos

- Conceitos básicos de programação
- Algoritmos, variáveis simples e tipos de variáveis
- Entrada e saída de dados
- Expressões lógicas
- Operações matemáticas
- Comandos condicionais e repetitivos
- Cadeias de caracteres / strings
- Tuplas, listas e dicionários
- Funções e arquivos

Por que aprender a programar?

- Neste curso vocês aprenderão o básico para se criar programas para computador.
- Exemplos de programas: Firefox , WhatsApp, MatLab, Media Player.
- Aprender a programar é uma atividade básica de um cientista, engenheiro, ou profissional no momento atual e futuro.

Organização básica de um ambiente computacional

Um ambiente computacional é organizado como uma hierarquia de funções, onde cada uma é responsável por uma tarefa específica.

Programas de Aplicação
Compiladores/Interpretadores
Sistema operacional
Hardware

Linguagens de Programação



PASCAL



Java™

SWI Prolog



Ruby



Linguagens de Programação

- Linguagem que define a sequência de instruções para processar dados em informações.
- Normalmente utilizamos uma **IDE** (Integrated Development Environment) para ajudar na escrita do código, achar erros, e executar o programa.
- Dividido em paradigmas:
 - Estrutural, Orientado a Objetos, Funcional, Lógico, entre outros.

Organização básica de um ambiente computacional

Compiladores e Linguagens de Programação.

- ❖ Uma linguagem de programação é um conjunto de comandos que são mais “próximos” da linguagem humana do que sinais digitais.
- ❖ O *compilador* é um programa que lê um código de uma linguagem de programação e o transforma em um programa executável.
- ❖ No caso de Python, ele é compilado em bytecode e, em seguida, executado pela respectiva Máquina Virtual (VM)

```
for(i=0; i< 10; i++)  
    c = a+b;
```

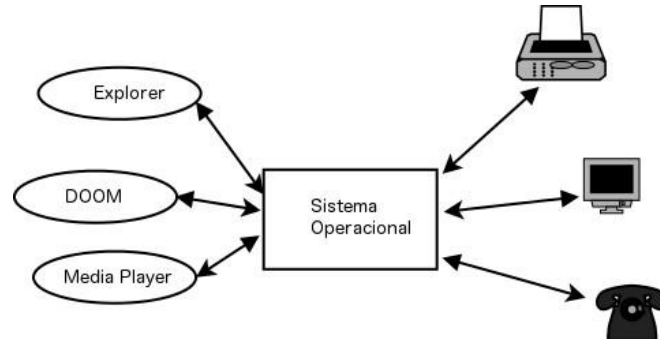
```
<code object f at 0x000001A1E093E660, file  
"<ipython-input-61-88c7683062d9>", line 1>
```

```
b'e\x00d\x00\x83\x01F\x00d\x01S\x00'
```


Organização básica de um ambiente computacional

Sistema Operacional.

- ❖ Os programas possuem instruções que são executados no hardware. Mas o acesso ao hardware, como disco rígido, memória e processador é controlado por um software especial conhecido como sistema operacional.
- ❖ O *sistema operacional* é o responsável pelo controle de hardware, segurança dentre outros.
- ❖ Exemplos de sistema operacional: *Windows, Mac OS, Linux, Android, iOS*.



Quais são os passos para fazer um sanduíche?

Vamos imaginar que vocês têm uma receita para fazer um sanduíche. Siga os passos na ordem correta e voilà, temos um sanduíche pronto!

Algoritmos

Ao criarmos um programa para realizar uma determinada tarefa devemos ser capazes de construir algoritmos.

- Algoritmo: Seqüência de passos, precisos e bem definidos, para a realização de uma tarefa.
- Algoritmos podem ser especificados de várias formas, inclusive em português.

Exemplo de algoritmo

Como calcular $5!$ (cinco fatorial) usando lápis, papel e uma tabuada?

- Sequência finita de passos bem definidos e não ambíguos
- Entrada bem especificada
- Devem ser executados em um intervalo de tempo finito
- Deve terminar

De algoritmos a programas

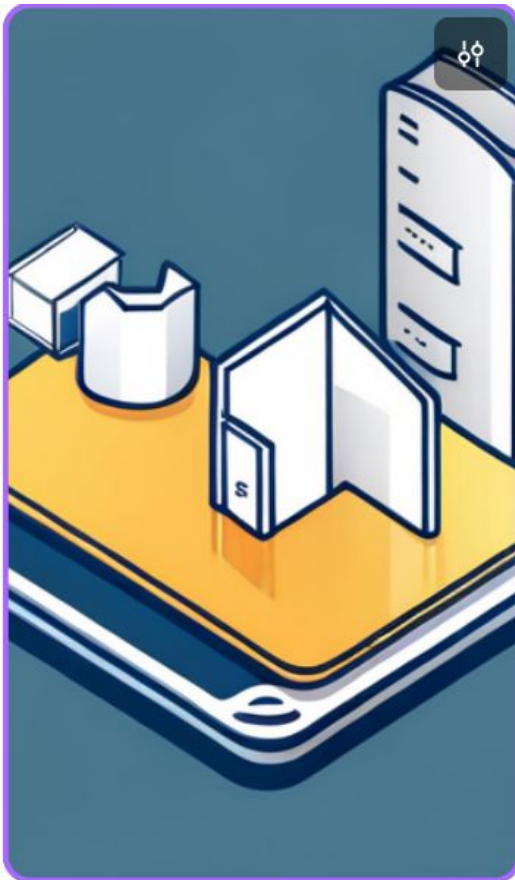
- ❖ Neste curso vamos aprender a criar algoritmos simples. Usaremos inicialmente a linguagem **Python** para descrever o algoritmos.
- ❖ Assim, teremos um programa para resolver a tarefa especificada.

Programação em Python

Python é uma linguagem de alto nível e de propósito geral que é fácil de aprender.

O código Python é fácil de ler e entender em comparação com outras linguagens.

Python tem uma sintaxe muito simples e limpa que se assemelha ao inglês do dia a dia.



História da Linguagem Python

A linguagem de programação Python foi criada por Guido van Rossum em 1989. Inicialmente, Python foi desenvolvida como uma resposta à linguagem ABC. A filosofia de design subjacente à Python é destacada pelo **mantra**: *'Bonito é melhor que feio, explícito é melhor que implícito, simples é melhor que complexo.'*

Principais características do Python

Linguagem Interpretada

Python é uma linguagem interpretada
Isso permite um rápido desenvolvimento

Indentação e Legibilidade

Python usa indentação ao invés de colchetes
Isso torna o código bem legível

Digitação Dinâmica

As variáveis têm tipos dinâmicos.

Os tipos são inferidos automaticamente

Principais características do Python

Estruturas básicas de código Python

Variáveis para armazenar dados
Declarações condicionais como *if/else* loops como *for*

Execução de código Python

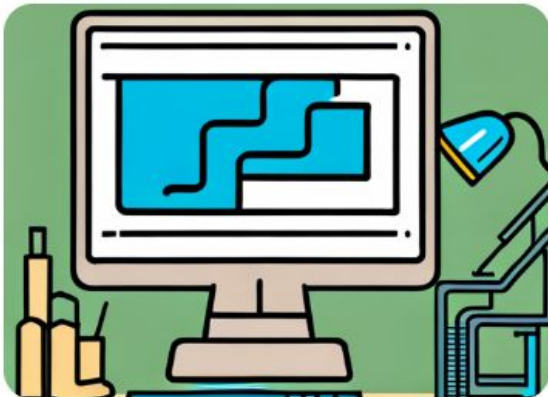
O código Python é interpretado linha por linha

Não é necessário compilar o código-fonte

Capacidades do Python

Vasta biblioteca
padrão Milhares de
pacotes para diversos
propósitos

Principais características do Python



Características Principais

O Python possui uma ampla biblioteca padrão. O código Python pode ser facilmente distribuído.

Versátil e com ampla adoção nas indústrias.



Estilos de Programação

Programação Orientada a Objetos
Programação Estruturada



Usos

Amplamente utilizado em desenvolvimento de software, ciência de dados, análise, automação, web e muito mais.

Torna-se uma habilidade valiosa para profissionais em diversas áreas.

Primeiros passos com Python

Editor On-line para praticar
programação.

<https://www.onlinegdb.com/>

Language: Python 3

Principais características do Python

Declaração de Variáveis

- ❑ **int** - inteiros: -3, -2, -1, 0, 1, 2, 3...
- ❑ **float** - reais: -3.2, -2.0, 1e5...
- ❑ **bool** - booleanos: True e False (Verdadeiro e Falso)
- ❑ **complex** - complexos: -2.0 + 3.4j
- ❑ **str** - cadeias de caracteres: 'A', 'B', 'LP01', 'Pode ser uma frase completa'
- ❑ **Outros tipos**

Tipos de Variáveis Inteiras

As variáveis **int** são o tipo de inteiro mais comumente usado.

As variáveis **unsigned int** só podem armazenar valores positivos.

short int usa 16 bits e tem um alcance menor que **int**.

Variáveis reais

O tipo float tem um alcance maior que int normal, usando **64 bits** para representar números reais grandes.

Variáveis

```
# Exemplo: Declaração de Variáveis
```

```
name = "Alice"
```

```
age = 25
```

```
# Exemplo: Tipos de Dados
```

```
height = 1.75
```

```
is_student = True
```

python

 Copy code

```
# Exemplo: Declaração de Variáveis
```

```
name = "Alice"
```

```
age = 25
```

```
# Exemplo: Tipos de Dados
```

```
height = 1.75
```

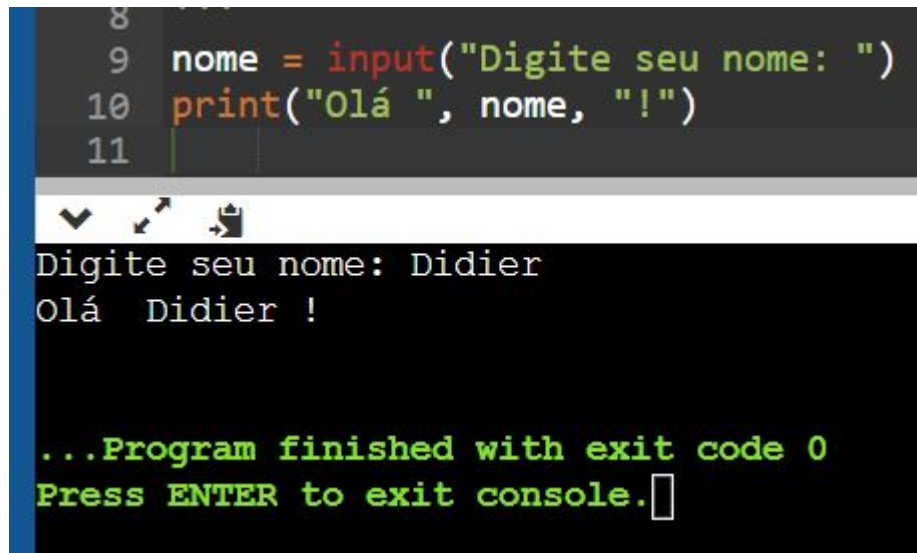
```
is_student = True
```

input é uma função para entrada de dados. Permite informar uma cadeia de caracteres na tela para o usuário, e retorna uma cadeia de caracteres digitada pelo usuário.

print é uma função para saída de dados. Permite mostrar ao usuário qualquer tipo de dados, separados por vírgulas.

Entrada e saída de dados

```
nome = input("Digite seu nome: ")  
print("Olá ", nome, "!")
```



The image shows a code editor window with a dark background and a terminal window below it. The code editor displays the following Python code:

```
8  
9 nome = input("Digite seu nome: ")  
10 print("Olá ", nome, "!")  
11
```

The terminal window shows the output of the program:

```
✓ ↗ 📄  
Digite seu nome: Didier  
Olá  Didier !  
  
...Program finished with exit code 0  
Press ENTER to exit console. □
```

Variáveis com input/output

Exemplo: Atribuição

x = 5

y = 10

sum = x + y

print(x,"+", y, "=", sum) # Saída: 15

```
8  
9  # Exemplo: Atribuição  
10 x = 5  
11 y = 10  
12 sum = x + y  
13 print(x,"+", y, "=", sum) # Saída: 15  
14
```

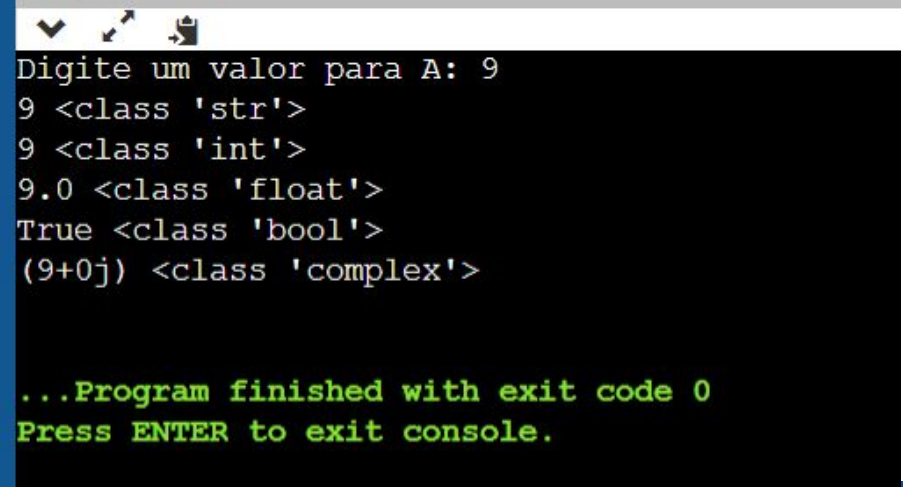
5 + 10 = 15

...Program finished with exit code 0
Press ENTER to exit console.

Variáveis com input/output

```
A = input("Digite um valor para A: ")
B = int(A)
C = float(A)
D = bool(A)
E = complex(A)
print(A, type(A))
print(B, type(B))
print(C, type(C))
print(D, type(D))
print(E, type(E))
```

```
8
9  A = input("Digite um valor para A: ")
10 B = int(A)
11 C = float(A)
12 D = bool(A)
13 E = complex(A)
14 print(A, type(A))
15 print(B, type(B))
16 print(C, type(C))
17 print(D, type(D))
18 print(E, type(E))
19
```



The terminal window shows the execution of the Python script. It starts with a prompt 'Digite um valor para A: 9'. Below this, it displays the type of each variable: '9 <class 'str'>', '9 <class 'int'>', '9.0 <class 'float'>', 'True <class 'bool'>', and '(9+0j) <class 'complex'>'. At the bottom, it shows the program finished with exit code 0 and a prompt to press ENTER to exit the console.

```
Digite um valor para A: 9
9 <class 'str'>
9 <class 'int'>
9.0 <class 'float'>
True <class 'bool'>
(9+0j) <class 'complex'>
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

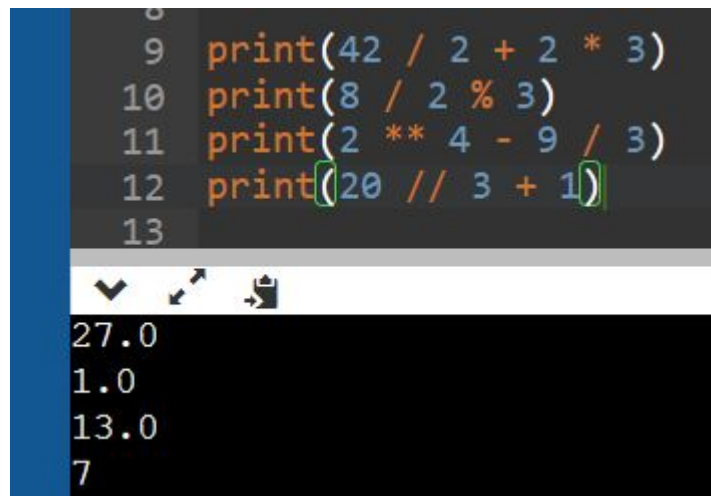

Mesma precedência da matemática:

1. parênteses
2. potência **
3. multiplicação *, divisão real /, divisão inteira //
4. resto da divisão inteira %
5. soma + e subtração -

Operações aritméticas

```
print(42 / 2 + 2 * 3)
print(8 / 2 % 3)
print(2 ** 4 - 9 / 3)
print(20 // 3 + 1)
```

```
9 print(42 / 2 + 2 * 3)
10 print(8 / 2 % 3)
11 print(2 ** 4 - 9 / 3)
12 print(20 // 3 + 1)
13
```

A screenshot of a Python IDE interface. The top part shows a code editor with four lines of Python code. The bottom part shows the output of the code execution. The code is: 9 print(42 / 2 + 2 * 3), 10 print(8 / 2 % 3), 11 print(2 ** 4 - 9 / 3), 12 print(20 // 3 + 1), 13. The output is: 27.0, 1.0, 13.0, 7. The code editor has a dark background with syntax highlighting. The output area is a separate window below the code editor.

27.0
1.0
13.0
7

Operações aritméticas

```
# Exemplo: Operações Aritméticas  
price = 100  
tax = 0.15  
total_price = price * (1 + tax)
```

```
8  
9 # Exemplo: Operações Aritméticas  
10 price = 100  
11 tax = 0.15  
12 total_price = price * (1 + tax)  
13 print(total_price)
```

114.99999999999999

Exercício 0: Algoritmo para uma tarefa comun

Proponha um algoritmo ou sequência de passos lógicos para realizar alguma tarefa do dia a dia.

Ex, fazer um suco de laranja, um sanduíche, etc.

Exercício 1: *Cálculo de Área de um Retângulo*

Escreva um código em Python que solicite ao usuário a largura e a altura de um retângulo e calcule a área desse retângulo. Em seguida, exiba o resultado na tela.

Exercício 2: *Conversão de Temperatura*

Crie um programa que converta uma temperatura em graus Celsius para Fahrenheit. O programa deve receber a temperatura em Celsius como entrada do usuário e exibir a temperatura equivalente em Fahrenheit.

Exercício 3: *Calculadora de IMC (Índice de Massa Corporal)*

Desenvolva um programa que calcule o IMC de uma pessoa. O usuário deve fornecer o peso (em quilogramas) e a altura (em metros). O programa deve calcular o IMC e exibir na tela.

Exercício 4: *Calculadora de Desconto em Compras*

Crie um programa que simule uma calculadora de desconto em compras. O usuário deve inserir o preço original do produto e o percentual de desconto a ser aplicado. O programa deve calcular o valor final após o desconto e exibir na tela.

Exercício 5: *Troca de Horas e Minutos*

Crie um programa que permita ao usuário inserir uma quantidade de minutos e converta essa quantidade em horas e minutos. O programa deve fazer a conversão usando a divisão inteira e o operador de resto da divisão (módulo), sem o uso de condicionais. Em seguida, o programa deve exibir o resultado na forma "X horas e Y minutos".

Por exemplo, se o usuário inserir 150 minutos, o programa deve exibir "2 horas e 30 minutos".

Dica: Use a divisão inteira para calcular as horas e o operador de resto da divisão para calcular os minutos restantes.

Programação de computadores

Objetivos

- Revisão de conteúdo anterior
- Operações matemáticas
- Expressões lógicas
- Comandos condicionais

Revisão Exercícios

Conversão temperatura de celsius para Fahrenheit

```
celsius = float(input("Digite graus Celsius: "))  
print("Fahrenheit: ",celsius*9/5+32)
```

```
1 celsius = float(input("Digite graus Celsius: "))  
2 print("Fahrenheit: ",celsius*9/5+32)
```

in

```
Digite graus Celsius: 0  
Fahrenheit: 32.0
```

Cálculo de IMC

```
peso = float(input("Digite o peso em kg: "))  
altura = float(input("Digite altura em m: "))
```

```
IMC = peso/altura**2  
print("IMC: ",IMC)
```

```
1 peso = float(input("Digite o peso em kg: "))  
2 altura = float(input("Digite altura em m: "))  
3 IMC = peso/altura**2  
4 print("IMC: ",IMC)
```

in

```
Digite o peso em kg: 60  
Digite altura em m: 1.65  
IMC: 22.03856749311295
```

Revisão Exercícios

Desconto em compras

```
preco = float(input("Digite o preço original: "))
desconto = float(input("Digite o desconto: "))
```

```
total = preco*(1.0 - desconto/100)
print("Preço com desconto: ",total)
```

```
1 preco = float(input("Digite o preço original: "))
2 desconto = float(input("Digite o desconto: "))
3 total = preco*(1.0 - desconto/100)
4 print("Preço com desconto: ",total)
```

input

```
Digite o preço original: 500
Digite o desconto: 10
Preço com desconto: 450.0
```

Exemplo de cálculo de horas e minutos

```
total_minutes = int(input("Digite a quantidade de minutos: "))
hours = total_minutes // 60
minutes = total_minutes % 60
print(f"{hours} horas e {minutes} minutos")
```

```
1 total_minutes = int(input("Digite a quantidade de minutos: "))
2 hours = total_minutes // 60
3 minutes = total_minutes % 60
4 print(f"{hours} horas e {minutes} minutos")
```

input

```
Digite a quantidade de minutos: 150
2 horas e 30 minutos
```

Declaração de variáveis

Regras para nomes de variáveis:

Devem começar com uma letra (maíuscula ou minúscula) ou Subscrito (_).

Nunca pode começar com números.

Pode conter letras maíusculas, minúsculas, números e subscrito.

Não pode-se utilizar como parte do nome de uma variável:

{ (+ - * / \ ; . , ?

Letras maíusculas e minúsculas são diferentes: Ex.

casa

Casa

Declaração de variáveis

Comando de Atribuição:

O comando serve para atribuir valores para as variáveis.

Letras maiúsculas e minúsculas são diferentes:

```
c = 10
```

```
C = 20
```

```
print(c)
```

à esquerda do operador de atribuição deve existir a **variável** que recebe o valor.

=

à direita, deve haver uma **expressão** cujo valor será calculado e armazenado na variável

Ex.

```
a = 1
```

```
a = '1'
```

```
a = "1"
```

```
a = 1.0
```

```
a = bool(1)
```

Expressões aritméticas

$\langle \text{expressao} \rangle + \langle \text{expressao} \rangle$: Calcula a soma de duas expressões. Ex: $a + b$

$\langle \text{expressao} \rangle - \langle \text{expressao} \rangle$: Calcula a subtração de duas expressões. Ex: $a - b$

$\langle \text{expressao} \rangle * \langle \text{expressao} \rangle$: Calcula o produto de duas expressões. Ex: $a * b$

$\langle \text{expressao} \rangle / \langle \text{expressao} \rangle$: Calcula a divisão de duas expressões. Ex: a / b

$\langle \text{expressao} \rangle // \langle \text{expressao} \rangle$: Calcula a parte inteira da divisão de duas expressões.

Ex: a / b

$\langle \text{expressao} \rangle \% \langle \text{expressao} \rangle$: Calcula o resto da divisão (inteira) de duas expressões.

Ex: $a \% b$

$- \langle \text{expressao} \rangle$: Inverte o sinal da expressão. Ex: $-b$

Expressões aritméticas

Mais sobre o operador resto da divisão: %

- Quando computamos “**a** dividido por **b**”, isto tem como resultado um valor **p** e um resto **r** < **b** que são únicos tais que

$$a = p * b + r$$

- Ou seja, **a** pode ser dividido em **p** partes inteiras de tamanho **b** e sobrar um resto **r** < **b**.

Exemplos:

- 5%2 tem como resultado o valor 1.
- 15%3 tem como resultado o valor 0.
- 1%5 tem como resultado o valor 1.
- 19%4 tem como resultado o valor 3.
- 5//2 tem como resultado o valor 2.
- 15//3 tem como resultado o valor 5.
- 1//5 tem como resultado o valor 0.
- 19//4 tem como resultado o valor 4.

Expressões aritméticas

Nos exemplo abaixo, quais valores serão impressos?

a) `print(27%3)`

b) `print(4//15)`

c) `print(27//3)`

d) `print(4%15)`

e) `print(10/3)`

f) `print(10//3)`

Podemos compor expressões complexas como por exemplo: `a = b + 2 + c+(9+d*8)`

g) `print(5 + 10 % 3)` – dá 0 ou 6 ???

Expressões aritméticas

Mesma precedência da matemática:

1. parênteses
2. potência **
3. multiplicação *, divisão real /, divisão inteira //
4. resto da divisão inteira %
5. soma + e subtração -

h) `print(5 * 10 % 3)` – dá 5 ou 2?

Expressões aritméticas

Atribuições simplificadas:

Comando	Exemplo	Corresponde a:
+=	$a += b$	$a = a + b$
-=	$a -= b$	$a = a - b$
*=	$a *= b$	$a = a * b$
/=	$a /= b$	$a = a / b$
%=	$a \% = b$	$a = a \% b$

Operações lógicas

Operações lógicas são realizadas sobre valores booleanos ou na comparação de expressões.

Comparações entre expressões aritméticas resultam em valores booleanos:

False, ou Zero (0), se o resultado é falso

True, ou qualquer outro número diferente de zero, se o resultado é verdadeiro.

A precedência nas operações lógicas é pela expressão aritmética seguida da expressão lógica.

Operadores Relacionais

Os operadores relacionais são:

`==` : igualdade.

`!=` : diferente.

`>` : maior que.

`<` : menor que.

`>=` : maior ou igual que.

`<=` : menor ou igual que.

Operadores Relacionais

< expressao > **==** < expressao >: Retorna verdadeiro quando as expressões forem iguais.

Ex: a **==** b

< expressao > **!=** < expressao >: Retorna verdadeiro quando as expressões forem diferentes. Ex: a **!=** b

< expressao > **>** < expressao >: Retorna verdadeiro quando a expressão da esquerda tiver valor maior que a expressão da direita. Ex: a **>** b

< expressao > **<** < expressao >: Retorna verdadeiro quando a expressão da esquerda tiver valor menor que a expressão da direita. Ex: a **<** b

< expressao > **>=** < expressao >: Retorna verdadeiro quando a expressão da esquerda tiver valor maior ou igual que a expressão da direita. Ex: a **>=** b

< expressao > **<=** < expressao >: Retorna verdadeiro quando a expressão da esquerda tiver valor menor ou igual que a expressão da direita. Ex: a **<=** b

Operadores Relacionais

Exemplos:

- a) `print( 9 <= 3 )`
- b) `print( (3*4)/2 == (2*3) )`
- c) `a = 1; b = -1; print( a != b )`
- d) `c = 10 != 9; print(c)`

Operadores lógicos

Expressões lógicas são aquelas que realizam uma operação lógica (**ou**, **e**, **não**, etc...) e retornam verdadeiro ou falso (como as expressões relacionais).

Operadores Lógicos

Em python temos os seguintes operadores lógicos:

and: operador E.

or: operador OU.

not: operador Não.

Expressões lógicas

- `< expressao > and < expressao >`: Retorna verdadeiro quando ambas as expressões são verdadeiras.

Dada a expressão A e B, temos a seguinte tabela de verdade:

A	B	A and B
V	V	V
V	F	F
F	V	F
F	F	F

Qual o resultado da expressão lógica abaixo?

```
a = 0; b = 0;
```

```
print( a == 0 and b == 0)
```


Expressões lógicas

- `< expressao > or < expressao >`: Retorna verdadeiro quando pelo menos uma das expressões é verdadeira. Sua tabela verdade é:

A	B	A or B
V	V	V
V	F	V
F	V	V
F	F	F

Qual o resultado da expressão lógica abaixo?

```
a = 0;
```

```
b = 1;
```

```
print(a == 0 or b == 0)
```

Expressões lógicas

- **not** < *expressao* >: Retorna verdadeiro quando a expressão é falsa e vice-versa. Sua tabela verdade é:

A	not A
V	F
F	V

Qual o resultado da expressão lógica abaixo?

a = 0;

b = 1;

print(not (a == 0 or b == 0))

Expressões lógicas

Exemplo

```
1 A = True
2 B = False
3 C = True
4 D = A or B
5 E = A and B
6 F = A and C or B
7 G = not C
8 print(D, E, F, G)
```

True False True False

Expressões lógicas

Exemplo

Considere as seguintes expressões lógicas: A e B. Determine se as expressões A e B são equivalentes.

Expressões:

A: $(P \text{ AND } Q) \text{ OR } (\text{NOT } P)$

B: $(P \text{ OR NOT } Q) \text{ AND } (\text{NOT } P \text{ OR } Q)$

Expressão A: $(P \text{ AND } Q) \text{ OR } (\text{NOT } P)$

P	Q	$(P \text{ AND } Q)$	$(\text{NOT } P)$	$(P \text{ AND } Q) \text{ OR } (\text{NOT } P)$
V	V	V	F	V
V	F	F	F	F
F	V	F	V	V
F	F	F	V	V

Expressão B: $(P \text{ OR NOT } Q) \text{ AND } (\text{NOT } P \text{ OR } Q)$

P	Q	$(P \text{ OR NOT } Q)$	$(\text{NOT } P \text{ OR } Q)$	$(P \text{ OR NOT } Q) \text{ AND } (\text{NOT } P \text{ OR } Q)$
V	V	V	V	V
V	F	V	F	F
F	V	F	V	F
F	F	V	V	V

Expressões lógicas

Exemplo

Considere as seguintes expressões lógicas: A e B. Determine se as expressões A e B são equivalentes.

Expressões:

A: NOT (NOT P AND Q) AND (P OR Q)

B: P

Expressão A: NOT (NOT P AND Q) AND (P OR Q)

P	Q	NOT P	NOT Q	NOT (NOT P AND Q)	P OR Q	NOT (NOT P AND Q) AND (P OR Q)
V	V	F	F	V	V	V
V	F	F	V	V	V	V
F	V	V	F	F	V	F
F	F	V	V	V	F	F

Expressões lógicas equivalentes

- $!(a == b)$ é equivalente a: $(a != b)$
- $!(a != b)$ é equivalente a: $(a == b)$
- $!(a > b)$ é equivalente a: $(a <= b)$
- $!(a < b)$ é equivalente a: $(a >= b)$
- $!(a >= b)$ é equivalente a: $(a < b)$
- $!(a <= b)$ é equivalente a: $(a > b)$

Comando condicional

Comando usado para direcionar a lógica do programa para executar um parte do código caso obedeça a uma condição especificada.

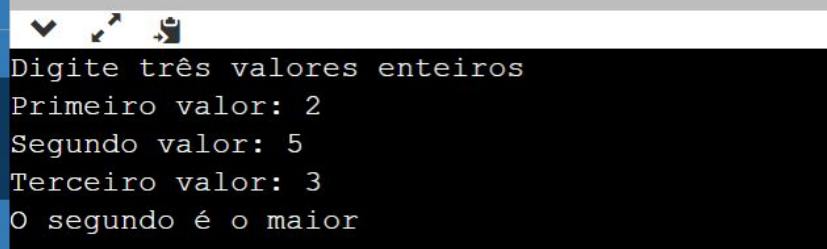
```
if (expressão lógica 1):  
    Bloco 1 código  
elif (expressão lógica 2):  
    Bloco 2 código  
else:  
    último bloco de código
```

Comando condicional

Exemplo:

Programa que recebe três números inteiros e retorna em qual posição está o maior número.

```
1 print("Digite três valores inteiros")
2 a = int(input("Primeiro valor: "))
3 b = int(input("Segundo valor: "))
4 c = int(input("Terceiro valor: "))
5 if a > b and a > c:
6     print("O primeiro é o maior")
7 elif b > a and b > c:
8     print("O segundo é o maior")
9 else:
10    print("O Terceiro é o maior")
11
```

The image shows a Python IDE window. The top part contains the Python code for finding the maximum of three numbers. Below the code editor, there is a toolbar with icons for running, debugging, and other IDE functions. The bottom part of the window shows the output of the program, which is the result of running the code with the input values 2, 5, and 3.

```
Digite três valores inteiros
Primeiro valor: 2
Segundo valor: 5
Terceiro valor: 3
O segundo é o maior
```